Back-References

When capturing sub-patterns are defined, they will automatically create (when matching) something called a *back-reference*. These are made available for use in the rest of the pattern using an escape sequence (though of course they are only available *after* the sub-pattern is closed). If you remember from earlier, I recommended that octal escapes used all three digits – `\001` instead of `\1` – and promised you an explanation to come. Well, here we are because the syntax for using back-references in an expression is simply the ordinal number of the sub-pattern prefixed with an escaping backslash. `\1` in a pattern would mean "octal value of 1" unless a sub-pattern is used, in which case it means "whatever was captured just now with the sub-pattern". Confused? Regexes can certainly get that way. Here's an example:

```
$text = 'hello hello world world';
preg_match_all('/(\w+)\s+\1/', $text, $matches);
print_r($matches);
```

```
Output:
Array
(
    [0] => Array
        (
            [0] => hello hello
            [1] => world world
        )

    [1] => Array
        (
            [0] => hello
            [1] => world
        )

)
```

The code above shows a common use case for back-references: finding repeating words. Let's break the pattern down. The sub-pattern `(\w+)` matches and captures one or more word characters (and the match is greedy so that entire words are captured), `\s+` matches one or more space characters, and finally `\1` to match whatever was captured